

데이터베이스 과제 최종 발표

미니 당근마켓

중고 물품 거래 플랫폼 — PostgreSQL + FastAPI 구현

릴레이션 4개

쿼리 3종

트랜잭션

Python 3.13

FastAPI

PostgreSQL 18

psycopg2

프로젝트 개요

상품 목록

카테고리·가격·제목·판매자 필터
검색

상품 등록

판매자·카테고리 선택, 제목·가
격 입력

구매 처리

트랜잭션으로 상태변경 + 거래
기록 동시

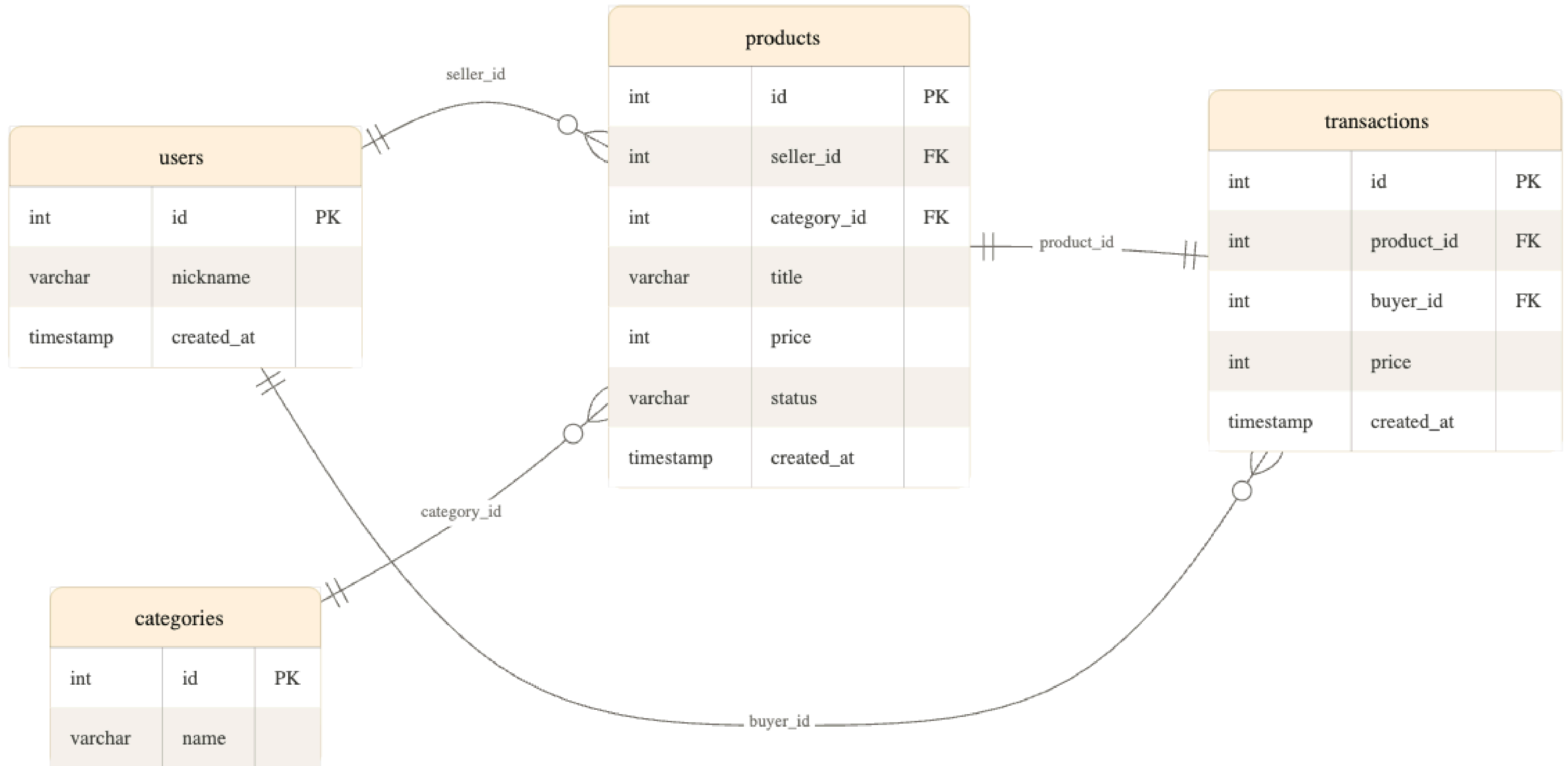
통계 조회

카테고리별·판매자별 집계, 평균
초과 상품

기술 스택 & 파일 구성

<code>main.py</code>	FastAPI 라우트 (API 엔드포인트)
<code>database.py</code>	psycopg2 연결·커서 관리
<code>models.py</code>	Pydantic 요청/응답 스키마
<code>schema.sql</code>	DDL + 더미 데이터
<code>queries.sql</code>	핵심 쿼리 + 트랜잭션 SQL
<code>index.html</code>	프론트엔드 (바닐라 HTML/JS)

릴레이션 설계 — ERD



ERD 설명

테이블 구성 요약

테이블	주요 컬럼	역할 설명
users	id (PK), nickname, created_at	판매자(seller) 및 구매자(buyer) 역할 동시 수행
categories	id (PK), name	상품 분류 기준 테이블, products와 1:N 관계
products	id (PK), seller_id (FK), category_id (FK), title, price, status	핵심 엔터티. users·categories 참조, transactions와 1:1 연결
transactions	id (PK), product_id (FK, UNIQUE), buyer_id (FK), price	거래 기록 테이블. product_id UNIQUE → 상품당 거래 1건

관계(Relationship) 요약

From	To	카디널리티	FK / 비고
users	products	1 : N	products.seller_id
categories	products	1 : N	products.category_id
products	transactions	1 : 1	transactions.product_id (UNIQUE)
users	transactions	1 : N	transactions.buyer_id

핵심 포인트

① users 이중 역할

users 테이블 하나로
판매자(seller_id)와
구매자(buyer_id) 모두 표현

② product_id UNIQUE

transactions.product_id에
UNIQUE 제약 → 하나의 상품은
단 1건의 거래만 허용

③ products 중심 설계

products가 users·categories·
transactions 모두와 연결되는
허브 엔터티

④ 정규화 수준

중복 없이 각 엔터티 역할 분리.
categories 분리로 상품 분류
관리 용이

쿼리 — JOIN / GROUP BY / CTE 서브쿼리

Q1. JOIN — 상품 목록 + 동적 필터

```
SELECT p.id, p.title, p.price,
       c.name AS category,
       u.nickname AS seller
FROM products p
JOIN users u      ON p.seller_id = u.id
JOIN categories c ON p.category_id = c.id
WHERE p.status = 'available'
      AND c.name = '전자기기'
      AND p.price BETWEEN 50000 AND 200000
ORDER BY p.created_at DESC;
```

3개 테이블 JOIN + 동적 WHERE 조합

Q2. GROUP BY — 카테고리별 통계

```
SELECT c.name AS category,
       COUNT(p.id) AS total,
       ROUND(AVG(p.price))::INT AS avg_price,
       MIN(p.price), MAX(p.price)
FROM categories c
LEFT JOIN products p ON c.id = p.category_id
GROUP BY c.id, c.name
ORDER BY total DESC;
```

ROUND()::INT 로 소수점 없이 정수 반환

Q3. CTE 서브쿼리 — 카테고리 평균보다 비싼 상품

```
WITH category_avg AS (
  SELECT category_id,
         ROUND(AVG(price))::INT AS avg_price
  FROM products GROUP BY category_id
)
SELECT p.title, p.price,
       ca.avg_price,
       p.price - ca.avg_price AS diff
FROM products p
JOIN category_avg ca USING (category_id)
WHERE p.price > ca.avg_price;
```

왜 CTE인가?

상관 서브쿼리: 행마다 3회 실행
→ $N \times 3$ 쿼리 발생

CTE: 카테고리 평균 1회만 계산
→ JOIN으로 재사용
→ 성능 + 가독성 개선

웹 서비스 ↔ DBMS 연동 구조

브라우저

index.html

fetch() / JSON

FastAPI

main.py

HTTP 라우팅

psycopg2

database.py

SQL 실행

PostgreSQL

DB

쿼리 결과

database.py — get_cursor()

```
@contextmanager
def get_cursor():
    conn = psycopg2.connect(**DB_CONFIG)
    conn.autocommit = False # 수동 트랜잭션
    cur = conn.cursor(
        cursor_factory=RealDictCursor
    )
    try:
        yield cur          # 요청 처리
        conn.commit()      # 정상 → COMMIT
    except Exception:
        conn.rollback()    # 오류 → ROLLBACK
        raise
    finally:
        cur.close()
        conn.close()      # 매 요청마다 닫음
```

핵심 설계 포인트

autocommit = False

모든 SQL을 트랜잭션 단위로 묶음.
BEGIN은 자동.

RealDictCursor

결과를 dict 반환 → FastAPI JSON 변환
에 편리.

요청마다 새 커넥션

전역 공유 대신 매 요청 독립 연결. 동시
요청 충돌 없음.

COMMIT / ROLLBACK

정상 → COMMIT, 예외 발생 →
ROLLBACK. 원자성 보장.

환경변수로 DB 설정

os.getenv()로 host·port·password 분
리. 보안.

트랜잭션 — 구매 처리

①

FOR UPDATE — 행 잠금

같은 상품 동시 구매 방지

②

UPDATE status = 'sold'

available일 때만 변경

③

rowcount 확인

0행 → 이미 sold → 409

④

INSERT transactions

거래 당시 가격 보존

COMMIT — 모두 성공

ROLLBACK — 하나라도 실패

main.py — purchase_product()

```
with get_cursor() as cur:

    # ① 행 잠금
    cur.execute(
        "SELECT price FROM products
        WHERE id=%s FOR UPDATE", (pid,)
    )

    # ② 상태 변경
    cur.execute(
        "UPDATE products SET status='sold'
        WHERE id=%s AND status='available'",
        (pid,)
    )

    # ③ 0행이면 이미 sold
    if cur.rowcount == 0:
        raise HTTPException(409,
            "이미 판매된 상품입니다")

    # ④ 거래 내역 INSERT
    cur.execute(
        "INSERT INTO transactions
        (product_id, buyer_id, price)
        VALUES (%s,%s,%s)",
        (pid, buyer_id, price)
    )

    # → 정상이면 자동 COMMIT
    # → 예외면 자동 ROLLBACK
```

트랜잭션 — 정상 vs 실패 케이스

정상 케이스 — 구매 성공

- 1 구매자가 [구매하기] 클릭
- 2 FOR UPDATE → product_id=1 행 잠금
- 3 UPDATE → status: available → sold (1행)
- 4 rowcount = 1 → 통과
- 5 INSERT INTO transactions (...) → 거래 기록
- 6 COMMIT → DB 반영 완료
- 7 HTTP 200 + 거래 정보 반환

실패 케이스 — 이미 판매된 상품

- 1 구매자 B가 같은 상품 [구매하기]
- 2 FOR UPDATE → 잠금 대기 또는 획득
- 3 UPDATE → status 이미 sold (0행)
- 4 rowcount = 0 → HTTPException(409)
- 5 ROLLBACK → INSERT 실행 안 됨
- 6 HTTP 409 → '이미 판매된 상품입니다'
- 7 DB 변경사항 없음 — 원자성 보장

UPDATE + INSERT 가 반드시 함께 성공하거나 함께 실패 — 원자성(Atomicity) 보장

프론트엔드 & API 연동

탭	API 엔드포인트	쿼리 종류	설명
상품 목록	GET /products	Q1 JOIN	카테고리.가격.제목.판매자 동적 필터
상품 등록	POST /products	INSERT	판매자.카테고리 FK 검증 후 등록
거래 내역	GET /transactions	Q4 JOIN	판매자.구매자 users 테이블 2회 JOIN
통계/카테고리	GET /stats/categories	Q2 GROUP BY	LEFT JOIN + 집계 함수
통계/판매자	GET /stats/sellers	Q2 CASE WHEN	조건부 집계
통계/평균초과	GET /stats/expensive	Q3 CTE	WITH절 서브쿼리
구매하기	POST /products/{id}/purchase	트랜잭션	FOR UPDATE + UPDATE + INSERT

단일 index.html — 탭 전환(CSS), fetch() API 호출, 동적 DOM 렌더링, 로딩 방지.본인구매 방지.에러 토스트 처리

구현 결과 & 실행 방법

실행 방법

```
# 1. PostgreSQL 시작
brew services start postgresql@18

# 2. 스키마 + 더미 데이터 적용
psql -d carrot_db -f schema.sql

# 3. 패키지 설치
pip install -r requirements.txt

# 4. 서버 실행
uvicorn main:app --reload

# 5. 브라우저 접속
http://localhost:8000      ← 프론트
http://localhost:8000/docs ← API 문서
```

구현 결과 요약

릴레이션

4개 테이블, FK 4개, 인덱스 4개

쿼리

JOIN / GROUP BY / CTE 3종

트랜잭션

FOR UPDATE + rowcount 체크 + ROLLBACK

API

엔드포인트 11개 (FastAPI)

프론트엔드

단일 index.html (바닐라 JS)

파일 구성

6개 파일 (py×3, sql×2, html×1)

마무리

이 과제에서 배운 것

01

릴레이션

테이블 설계는 코드 이전에 완성되어야 한다. FK·CHECK·UNIQUE 제약조건은 애플리케이션보다 DB 레벨에서 먼저 막는 것이 안전하다.

02

쿼리

JOIN 없이는 정규화된 DB에서 의미있는 데이터를 뽑기 어렵다. CTE는 같은 서브쿼리를 반복 실행하는 문제를 한 번에 해결한다.

03

트랜잭션

UPDATE와 INSERT를 별도로 실행하면 순간적으로 데이터 불일치가 생긴다. BEGIN~COMMIT으로 묶어야 원자성이 보장된다.

감사합니다